

VITA MODULAR · PROJECT HUB

Backend Specification

A technical brief for adding a shared backend so multiple users can read and edit the same hub data

Hub v1.0.0 · Schema v5 · May 2026

Contents

1. Brief	3
2. Recommended stack	5
3. Architecture overview	7
4. Database schema	8
5. File storage	12
6. Realtime updates	13
7. Frontend integration	15
8. Cutover plan	17
9. Deployment	18
10. Effort estimate & risks	20

1. Brief

The Vita Modular Project Hub is a single-file HTML application that today persists all data to the user's browser localStorage. It works well for a single user but cannot share state across multiple browsers or users — each person sees only their own copy.

This document specifies the changes needed to introduce a shared backend so that multiple users can access the same hub URL, see each other's edits in near real-time, and have a single source of truth for all project data. There is no requirement for individual user accounts or permissions: anyone with access to the hub URL has full read/write access.

1.1 What's already done

The frontend has been refactored to make this transition as smooth as possible:

- **A single datastore abstraction** wraps all read/write operations. The current implementation reads/writes localStorage. The backend swap is contained to this one object — the rest of the hub doesn't change.
- **All datastore methods are async** (return Promises) so the swap to a remote backend is transparent at every callsite.
- **Schema versioning** is in place with formal migrations from v1 through v5. The same schema definition will inform the database design.
- **Export/Import is implemented** and produces a single JSON file containing all hub data. This is the cutover mechanism — users export from localStorage, then the hub imports into the backend on first load.

1.2 What's required

Build a backend that:

1. Stores all hub data (projects, cost projects, team, settings) in a queryable database.
2. Stores binary files (quote PDFs, contracts, photos) in object storage with public-readable URLs.
3. Provides realtime updates — when one user makes a change, other open browsers update within seconds without manual refresh.
4. Is reachable from a frontend hosted on Netlify, with no individual login.
5. Has minimal hosting and operational complexity — this is a small business tool, not a SaaS platform.

1.3 What's explicitly not required

- **Per-user authentication** — all users share a single access path. Treat the hub URL itself as the access control boundary.
- **Granular permissions** — there are no roles. Anyone with the URL can read or write anything.
- **Audit logging** — who changed what is captured implicitly through the existing notes/works completion fields. No separate audit trail is required.
- **Conflict resolution beyond last-write-wins** — if two users edit the same project at once, the last save wins. Realtime updates will surface most conflicts to users before they happen.

Why this matters

Keeping the scope tight is important. A full multi-tenant auth system would add weeks of work for negligible benefit; the hub is used by a small co-located team who already trust each other. The simpler the backend, the cheaper to host and the easier to maintain.

2. Recommended stack

Two products, both already chosen for the frontend:

2.1 Netlify (frontend hosting)

Netlify already hosts the hub HTML. Continue using it. Atomic deploys, free SSL, custom domain support, instant rollback, and password-protect-the-whole-site is available on paid plans for additional perimeter security.

2.2 Supabase (database, file storage, realtime)

Supabase is an open-source backend-as-a-service. It bundles everything this project needs in a single managed product:

- **Postgres database** — managed, with full SQL access. The data model maps cleanly to relational tables, which keeps reports easy to write and the dataset queryable from any tool that speaks SQL.
- **Storage** — S3-compatible object storage for files (quote PDFs, contracts, photos). Public-readable buckets remove the need for signed URLs in this no-auth setup.
- **Realtime** — websocket subscriptions on table changes, broadcast to all connected clients. Built-in, no additional infrastructure.
- **JavaScript SDK** — talks to all of the above from the browser. Good docs, well-maintained.
- **Free tier** — 500 MB database, 1 GB file storage, 2 GB monthly bandwidth, unlimited API calls. Ample for the hub's expected data volume; the entire current export from a fully-loaded localStorage instance is around 10 MB.
- **Paid tier** — \$25/month for the Pro plan with 8 GB database, 100 GB file storage, daily backups, and 7-day point-in-time recovery. Worth the upgrade once production data lives there.
- **Open source** — the entire stack can be self-hosted on your own infrastructure if you ever want to migrate away. No vendor lock-in.

2.3 Alternatives considered

Option	Why not
Firebase / Firestore	NoSQL document store. Hub data is naturally relational (projects with notes, tasks, works items). Reports become harder to write. Vendor lock-in is more total than Supabase.
Netlify Blobs + Functions	Newer, more limited. No realtime out of the box. Would need to bolt on websockets separately. More code to write and maintain.
Custom Node API + Postgres on a VPS	Maximum flexibility but maximum operational overhead. Server provisioning, SSL renewal, backups, monitoring all become your problem. Disproportionate effort for this scale.
Airtable / Notion as backend	Tempting for low-code, but rate limits, no realtime in browser, and the data model is too rigid for the hub's nested structures (notes/tasks/works inside projects).

2.4 Access control

Since there are no individual users, the hub URL is the security boundary. Two complementary controls are recommended:

6. **Netlify password protection** — available on Netlify Pro (\$19/site/month) or Business plans. A single shared password protects the hub URL itself.
7. **Supabase Row Level Security policies** — anonymous (anon) role granted full read/write on all tables. The Supabase URL and anon key are embedded in the hub HTML; they are not secrets, but the Netlify password keeps the HTML itself off the open internet.

On security

This setup is appropriate for a small business tool used by a known team. It is not appropriate for handling personally-identifiable data of customers at scale, payment information, or anything subject to GDPR data subject rights without additional controls. The team should be aware that anyone who has the password and URL has unrestricted access.

3. Architecture overview

Three parties, all hosted services:

```

Browser (Vita team)
  ↓ HTTPS
Netlify (static HTML, password-protected)
  ↓ Supabase JS SDK
Supabase (Postgres + Storage + Realtime)

```

There is no custom server code. Netlify serves static files. The hub HTML talks directly to Supabase from the browser using the Supabase JavaScript SDK. Supabase handles the database, file storage, and realtime websocket subscriptions.

3.1 Why no custom backend

A common pattern is browser → custom API → database. Cutting out the custom API simplifies operations significantly: one fewer codebase to maintain, one fewer thing to deploy, one fewer place for bugs to live. Supabase provides a configurable HTTP API automatically from the database schema (PostgREST), so you get the API for free.

Trade-off: the database connection details (Supabase URL and anon key) live in the hub HTML, visible to anyone who views source. That is acceptable here because the URL is password-protected at the Netlify perimeter and the access model is uniform — no privileged user has a different key.

3.2 Data flow when a user makes a change

8. User edits a project field in the hub UI.
9. Hub calls `dataStore.saveProject(p)`.
10. The `dataStore` (now backed by Supabase) issues an `UPDATE` statement via the Supabase JS SDK over HTTPS.
11. Postgres applies the change and broadcasts it to all subscribed clients via the Realtime service.
12. Other users' hubs receive the realtime event, refresh their local in-memory copy, and re-render the affected views.

Latency end-to-end: typically 200-500ms within the UK/EU. Local optimistic updates in the originating user's UI make this invisible to them.

3.3 Offline behaviour

If a user's browser loses connection to Supabase, writes will fail. The hub should surface this clearly via the existing toast system and the topbar sync indicator. A future enhancement could queue writes locally and replay on reconnection, but is out of scope for v1 — the team is co-located with reliable connectivity.

4. Database schema

The schema below maps the existing localStorage data model to relational tables. Every JavaScript field in the current datastore methods has a corresponding column. Nested arrays (notes, tasks, works) are split into child tables with a foreign key back to the parent project.

All ID fields use TEXT (the existing IDs are JavaScript-generated strings, not numeric). Timestamps use TIMESTAMPTZ. JSON-shaped fields that have variable structure (ancillary items, ad-hoc settings) use JSONB.

4.1 projects

The core table. One row per project.

```
CREATE TABLE projects (
  id          TEXT PRIMARY KEY,
  ref         TEXT,
  name        TEXT,
  email       TEXT,
  phone       TEXT,
  address     TEXT,
  ext_type    TEXT,
  finish      TEXT,
  area        NUMERIC,
  rate        NUMERIC,
  base        NUMERIC,
  anc_items   JSONB DEFAULT '[]'::jsonb,
  anc_total   NUMERIC,
  discount_pct NUMERIC,
  discount_amt NUMERIC,
  grand_total NUMERIC,
  stage       TEXT,
  temperature TEXT DEFAULT 'cold',
  tracker_source TEXT,
  contract_signed BOOLEAN DEFAULT false,
  contract_signed_date TIMESTAMPTZ,
  contract_generated BOOLEAN DEFAULT false,
  quote_pdf_generated BOOLEAN DEFAULT false,
  install_start TIMESTAMPTZ,
  spec        JSONB,      -- the full spec object (kitchen, bath, etc.)
  created     TIMESTAMPTZ DEFAULT NOW(),
  modified    TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX ON projects (modified DESC);
CREATE INDEX ON projects (stage);
```

4.2 project_notes

Each project has many notes. Append-only audit log of free-text comments.

```
CREATE TABLE project_notes (
  id          TEXT PRIMARY KEY,
  project_id  TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  body       TEXT,
  author_id   TEXT,
  author_name TEXT,
  author_color TEXT,
  ts         TIMESTAMPTZ DEFAULT NOW()
);
```



```
CREATE INDEX ON project_notes (project_id, ts DESC);
```

4.3 project_tasks

Each project has many tasks. Open or done.

```
CREATE TABLE project_tasks (
  id          TEXT PRIMARY KEY,
  project_id  TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  title       TEXT,
  description TEXT,
  assignee_id TEXT,
  due_date    DATE,
  status      TEXT DEFAULT 'open', -- 'open' | 'done'
  created     TIMESTAMPTZ DEFAULT NOW(),
  created_by  TEXT,
  completed   TIMESTAMPTZ
);
CREATE INDEX ON project_tasks (project_id);
CREATE INDEX ON project_tasks (assignee_id) WHERE status = 'open';
CREATE INDEX ON project_tasks (due_date) WHERE status = 'open';
```

4.4 project_works

The works checklist. 38 items per project, populated from the VITA_WORKS_TEMPLATE in the hub. Each row tracks status and completion metadata for one checklist item.

```
CREATE TABLE project_works (
  id          TEXT PRIMARY KEY,
  project_id  TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  template_id TEXT NOT NULL,      -- 'wk-1' through 'wk-38'
  section_id  TEXT NOT NULL,      -- 'sales-handover', 'design', etc.
  label       TEXT,
  status      TEXT DEFAULT 'outstanding',
              -- 'outstanding' | 'inprogress' | 'complete' | 'na'
  notes       TEXT,
  conditional BOOLEAN DEFAULT false,
  milestone   TEXT,              -- 'payment' or NULL
  completed_date TIMESTAMPTZ,
  completed_by TEXT,
  UNIQUE (project_id, template_id)
);
CREATE INDEX ON project_works (project_id);
CREATE INDEX ON project_works (project_id, status);
```

4.5 project_files

Files attached to a project (quote PDFs, contracts, photos, drawings). The actual file bytes live in Supabase Storage; this table holds metadata and the storage URL.

```
CREATE TABLE project_files (
  id          TEXT PRIMARY KEY,
  project_id  TEXT NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  name        TEXT,
  kind        TEXT,              -- 'quote-pdf' | 'contract' | 'cost-tracker' | 'other'
  version     INTEGER,
  size_bytes  BIGINT,
  storage_url TEXT,              -- public Supabase Storage URL
```

```

    added      TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX ON project_files (project_id);

```

4.6 works_photos

Photos attached to works checklist items (snags, evidence, site access). Same pattern as project_files — bytes in Storage, metadata here.

```

CREATE TABLE works_photos (
  id          TEXT PRIMARY KEY,
  work_id     TEXT NOT NULL REFERENCES project_works(id) ON DELETE CASCADE,
  name        TEXT,
  size_bytes  BIGINT,
  storage_url TEXT,
  added       TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX ON works_photos (work_id);

```

4.7 cost_projects

Cost intelligence — separate from main projects. CVR / Final Account analysis with line items.

```

CREATE TABLE cost_projects (
  id          TEXT PRIMARY KEY,
  name        TEXT,
  area        NUMERIC,
  contract_value NUMERIC,
  actual_cost NUMERIC,
  hub_project_id TEXT,          -- optional FK to projects.id
  items       JSONB DEFAULT '[]'::jsonb,
  created     TIMESTAMPTZ DEFAULT NOW(),
  modified    TIMESTAMPTZ DEFAULT NOW()
);

```

4.8 team_members

The shared team list. One global table, no per-project membership.

```

CREATE TABLE team_members (
  id    TEXT PRIMARY KEY,
  name  TEXT NOT NULL,
  color TEXT
);

```

4.9 settings

Single-row key/value table for global hub settings (schedule capacity, current user id, sync metadata). Simpler than dedicated tables for one-row-each settings.

```

CREATE TABLE settings (
  key  TEXT PRIMARY KEY,
  value TEXT
);
-- Known keys:
-- 'schemaVersion' : '5'

```

```
-- 'schedCapacity'      : '3'
-- 'currentUserId'      : 'tm_abc123'
-- 'syncMeta'           : '{"lastSync":...}' (JSON-encoded)
```

4.10 Row Level Security policies

All tables get the same policy: anonymous access for SELECT, INSERT, UPDATE, DELETE. This relies on the Netlify password to be the actual access boundary.

```
ALTER TABLE projects ENABLE ROW LEVEL SECURITY;
CREATE POLICY "anon full access" ON projects
  FOR ALL TO anon
  USING (true) WITH CHECK (true);
-- Repeat for every table.
```

A note on schema migrations

Once the backend is live, schema changes follow the same versioned migration pattern as the hub already uses. Bump `SCHEMA_VERSION` in the hub, write a migration function, and run a corresponding `ALTER TABLE` / `UPDATE SQL` on Supabase. Keep them in lockstep — the hub assumes the schema version it knows about.

5. File storage

Today, the hub stores quote PDFs, contracts, and photos as base64 data URLs inside the project objects in localStorage. This works at small scale but bloats the database every time it's read or written. Object storage is the right home for binary blobs.

5.1 Buckets

Two buckets, both public-readable, both with anonymous write access (constrained by RLS as needed):

- **project-files** — quote PDFs, contracts, cost trackers, and any other files attached to a project.
- **works-photos** — photos attached to works checklist items.

Public-readable means anyone with the URL can fetch the file directly. This is acceptable because the URLs are unguessable (they include long random IDs) and access to find them is gated by the Netlify password on the hub itself.

5.2 Path conventions

Use the project ID as a folder so files for a deleted project can be cleaned up with a single recursive delete:

```
project-files/{project_id}/{file_id}-{filename}
works-photos/{project_id}/{work_id}/{photo_id}-{filename}
```

5.3 Upload flow

13. User selects a file in the hub UI (drag-drop, file input, or generated programmatically e.g. quote PDF).
14. Hub uploads to Supabase Storage via the JS SDK: `supabase.storage.from('project-files').upload(path, blob)`.
15. SDK returns the public URL.
16. Hub inserts a row into `project_files` (or `works_photos`) with name, kind, size, and the URL.
17. Realtime event broadcasts to other connected clients; their UI re-renders to show the new file.

5.4 Cleanup

When a project or works item is deleted, the corresponding storage objects must also be deleted — Postgres CASCADE only handles the rows, not Supabase Storage. Two options:

- **Frontend cleanup** — before DELETE-ing the row, list the related storage paths and delete each. Acceptable for v1 since deletion is rare.
- **Database trigger** — Supabase Edge Function on row delete that cleans up the corresponding storage. More robust, more setup.

6. Realtime updates

Supabase Realtime broadcasts row-level INSERT, UPDATE, and DELETE events to subscribed clients over websockets. The hub subscribes to every shared table on boot and updates its in-memory copy when events arrive.

6.1 Subscription setup

On boot, after the initial data load, set up subscriptions:

```
supabase.channel('hub-realtime')
  .on('postgres_changes',
    { event: '*', schema: 'public', table: 'projects' },
    handleProjectChange)
  .on('postgres_changes',
    { event: '*', schema: 'public', table: 'project_notes' },
    handleNoteChange)
  // ...one subscription per table
  .subscribe();
```

6.2 Handling events

Each handler updates the in-memory state and triggers a re-render. The `dataStore` gains an event hook to broadcast within-the-page that data has changed:

```
function handleProjectChange(payload) {
  const { eventType, new: row, old } = payload;
  if (eventType === 'INSERT' || eventType === 'UPDATE') {
    upsertLocalProject(row);
  } else if (eventType === 'DELETE') {
    removeLocalProject(old.id);
  }
  triggerRender(); // re-render whatever view is active
}
```

6.3 Origin suppression

Without care, every write a user makes will round-trip back to them as a realtime event, causing a redundant re-render. Filter these out by tagging writes with a per-tab session ID and ignoring events with a matching ID:

```
const sessionId = crypto.randomUUID();
// On every write, include the session ID in a metadata column.
// On every realtime event, ignore if metadata.session === sessionId.
```

Alternative: optimistic updates apply locally before the SDK call resolves, and the realtime echo is treated as a confirmation rather than a change.

6.4 What to subscribe to

Subscribe to every shared table the hub renders from. At minimum:

- projects
- project_notes
- project_tasks

- project_works
- project_files
- works_photos
- cost_projects
- team_members
- settings

7. Frontend integration

All frontend changes are localised to the `dataStore` object in the hub HTML. The rest of the codebase is unaffected.

7.1 The current `dataStore`

Today the `dataStore` wraps `localStorage`. Every method is async (returns a Promise). The full API:

Method	Purpose
<code>saveProject(p)</code>	Insert or update a project
<code>deleteProject(id)</code>	Remove a project
<code>loadAllProjects()</code>	Return all projects
<code>saveCostProject(p)</code>	Insert or update a cost project
<code>deleteCostProject(id)</code>	Remove a cost project
<code>loadAllCostProjects()</code>	Return all cost projects
<code>saveTeam(t)</code>	Replace the team list
<code>loadTeam()</code>	Return the team list
<code>getCurrentUserId()</code> / <code>setCurrentUserId(id)</code>	Active user
<code>getSetting(k)</code> / <code>setSetting(k, v)</code>	Generic settings
<code>getSyncMeta()</code> / <code>setSyncMeta(m)</code>	Sync metadata
<code>getSchemaVersion()</code> / <code>setSchemaVersion(v)</code>	Schema version
<code>runMigrations()</code>	Apply any pending migrations
<code>exportAll()</code> / <code>importAll(data, opts)</code>	Bulk export/import

7.2 The Supabase implementation

Replace the body of each method with a Supabase SDK call. Method signatures stay the same.

Example — `saveProject`:

```
async saveProject(p) {
  p.modified = new Date().toISOString();
  const { error } = await supabase
    .from('projects')
    .upsert(toSnakeCase(p));
  if (error) throw error;
}
```

`loadAllProjects`:

```
async loadAllProjects() {
  const { data, error } = await supabase
    .from('projects')
    .select(`
      *,
      project_notes (*),
      project_tasks (*),
    `);
  if (error) throw error;
  return data;
}
```

```

    project_works (*),
    project_files (*)
  `)
  .order('modified', { ascending: false });
  if (error) throw error;
  return data.map(toCamelCase);
}

```

7.3 Field name conversion

Postgres conventionally uses snake_case (contract_signed); JavaScript conventionally uses camelCase (contractSigned). Add a thin helper that converts on read and write so neither side needs to bend:

```

function toSnakeCase(obj) { /* contractSigned → contract_signed */ }
function toCamelCase(obj) { /* contract_signed → contractSigned */ }

```

Keep the JavaScript-side names exactly as they are today. The hub's render code, exports, and migrations all reference the camelCase names; changing them would ripple through the codebase unnecessarily.

7.4 Loading nested data efficiently

Supabase's nested-select syntax (above) returns a project plus all its notes/tasks/works/files in a single query. For the hub's typical scale (low hundreds of projects), this is fine. If the dataset grows past 1,000 projects, switch to lazy loading: fetch project rows on initial load, fetch notes/tasks/works on project open.

7.5 Realtime integration

Add a setupRealtime() method on datastore that subscribes to every shared table and dispatches updates back into the in-memory state. Call it from the hub's init() function once after loadAllProjects() completes.

8. Cutover plan

Existing users have data in their browser localStorage. Migrating them to the shared backend without losing data is a one-time exercise.

8.1 Sequence

18. Build and test the backend against synthetic data first.
19. Deploy a staging copy of the hub at a separate URL (e.g. hub-staging.vita-modular.example) pointed at a separate staging Supabase project.
20. Each existing user opens the current production hub and clicks Settings → Export everything. They save the JSON file.
21. They open the staging hub. The staging hub detects an empty database, prompts them to import, and they upload the JSON file.
22. Verify each user's data looks correct in staging. Reconcile any duplicates manually if multiple users had overlapping projects.
23. Promote the staging hub URL to be the new production URL (or simply update the production hub to point at the production Supabase project, having migrated the merged data over).
24. Each user is told to discard their old browser localStorage to avoid confusion. The Settings → Wipe all hub data button does this.

8.2 Merging multiple exports

If two or more users have made independent edits to overlapping data (same project IDs), a manual merge is required. The simplest approach:

25. Designate one user's export as the source of truth (whoever has been the most active in the hub recently).
26. Manually inspect other users' exports for projects that don't exist in the source-of-truth export. Merge those in by hand by editing the JSON.
27. For projects that exist in both with different contents, take the most recently modified one (the modified field is your guide).
28. Import the merged JSON into the production Supabase database.

This is a one-time exercise that should take 30-60 minutes for a team of 3-5 users with overlapping data.

8.3 The import flow

The hub already has `dataStore.importAll()` working. After the backend swap, `importAll()` will write to Supabase rather than localStorage, but the user-facing flow is unchanged: Settings → Import → choose file → done.

Communicate the cutover

Schedule a short call with the team. Walk through Export → Import together so everyone follows the same process. After cutover, confirm with each user that their projects look correct before instructing them to wipe their localStorage.

9. Deployment

9.1 Repository structure

Single Git repository, deployed via Netlify's GitHub integration. Auto-deploy on push to main:

```

hub/
├── index.html           # the hub itself
├── netlify.toml         # Netlify config (password protection, headers)
├── README.md           # setup notes for new contributors
└── docs/
    ├── User_Guide.pdf
    └── Backend_Specification.pdf (this document)
  
```

9.2 Netlify configuration

netlify.toml — site password and cache headers:

```

[build]
  publish = "."

[[headers]]
  for = "/index.html"
  [headers.values]
    Cache-Control = "no-cache, must-revalidate"

# Site-level password protection requires Netlify Pro/Business
# Enable via the Netlify dashboard, not via netlify.toml
  
```

9.3 Supabase configuration

- Create a project in the Supabase dashboard.
- Run the schema SQL from section 4 to create all tables and policies.
- Create the two storage buckets (project-files, works-photos) and set them public-readable.
- Copy the project URL and anon key into the hub HTML where indicated by a clearly-marked configuration block at the top of the script.
- Set up automatic backups on the Pro plan.
- Configure Realtime to be enabled on every shared table (default on the Free tier; verify on Pro).

9.4 Updating the hub after launch

Day-to-day updates are unchanged from the developer's perspective:

29. Edit hub/index.html locally.
30. Test against the staging Supabase project.
31. git push to main.
32. Netlify deploys the new HTML to production within 30 seconds.
33. Users refresh their browser and see the new version.

Schema changes follow the migration pattern. Bump `SCHEMA_VERSION` in the HTML, add a migration function to `SCHEMA_MIGRATIONS`, and run the matching SQL on Supabase before the new HTML rolls out.

9.5 Backups

Three layers, none of which are optional:

- **Supabase Pro automatic backups** — daily, retained 7 days, included in the Pro plan.
- **Manual exports** — weekly, via Settings → Export everything. Save to a shared Drive folder. The export captures every JSON-serialisable piece of state.
- **Pre-migration backups** — captured automatically by the hub before every schema migration. Recoverable from Settings → Restore.

10. Effort estimate & risks

10.1 Effort

Rough estimates for a developer familiar with JavaScript and SQL but new to Supabase. Add 20-30% for unfamiliarity:

Task	Estimate
Supabase project setup, schema SQL, RLS policies	0.5 day
Storage buckets and upload plumbing	1 day
dataStore Supabase implementation (every method)	3-4 days
snake_case / camelCase conversion helpers	0.5 day
Realtime subscriptions and event handlers	1-2 days
Origin suppression / optimistic updates	0.5-1 day
Testing with synthetic data, fix bugs	2-3 days
Cutover documentation and team walkthrough	0.5 day
Buffer for unknowns	2 days
Total	11-14 days (~3 weeks)

10.2 Risks

- **Realtime echo loops** — if origin suppression is missed, every write triggers a redundant re-render, with possible flicker. Test thoroughly with two browsers open side by side.
- **Storage cleanup gaps** — deleted projects can leave orphaned files in storage if the cleanup step is missed. Schedule a periodic audit (or implement the database trigger).
- **Free tier limits** — 500 MB database is generous but not infinite. Monitor usage; upgrade to Pro before hitting it.
- **Cutover data loss** — if a user forgets to export before wiping their localStorage, their work is gone. Communicate clearly and verify each user's data after import.
- **No conflict surface** — last-write-wins is acceptable for a small team but could surprise users who edit the same record offline. If this proves problematic, add a "modified by Alice 5 minutes ago" indicator on records.

10.3 Out of scope for v1

Things deliberately not in this brief that may be useful later:

- Per-user authentication and roles
- Audit logging beyond the existing notes/works metadata
- Offline mode with deferred writes
- Granular conflict resolution
- Public-facing read views (e.g. client-shareable project pages)
- Automated daily exports to a backup location

Each is a sensible enhancement once the v1 backend is proven in production.

10.4 Questions for the team

Before starting work, the developer should confirm:

34. Is the Netlify Pro plan budgeted for? (\$19/site/month for password protection.)
35. Is the Supabase Pro plan budgeted for? (\$25/month, recommended for production data.)
36. Is there a preferred staging URL convention for testing the new backend before cutover?
37. Who owns the cutover walkthrough call with users?
38. Who has admin access to the Netlify and Supabase dashboards once live?

Final note

The frontend has been deliberately structured to make this transition as low-risk as possible. The `dataStore` abstraction means every read/write callsite is already routing through one place. Schema versioning means the database design is already specified. Export/Import means the cutover mechanism is already built. The work is mostly mechanical: implement Supabase calls inside `dataStore` methods, set up subscriptions, run the cutover. Three weeks is a comfortable estimate.